编译原理 | 实验4 计算机学习助手基础功能升级与进阶功能开发

http://complier.freebacon.cn/

老师可以访问这个链接（github 上部署所以可能打开有点慢 or 开加速器）

5,6,7 为本次作业的添加项

一、实验目标

1. 巩固本次课**文法句柄、0–3型文法分类、词法分析基础**核心概念，完成智能学习助手的基础模块升级，实现自动解析与判定功能。
2. 深化句柄、四类文法的应用，实现简易词法分析工具，整合所有模块，形成最终版完整的计算机专业课智能学习助手。
3. 提交可运行程序 + 实验报告（功能截图+核心代码说明，≤10页 **任务 1**）+ 完整可运行程序 + 实验报告（全功能演示截图+核心算法讲解，≤10页 **任务 2**），二者合并提交 20 页以内。总共 7 项任务。

二、文法定义（与实验 3 一致）

对于 "词法分析和语法分析有什么关联吗？"，这句话我直接提取 ["词法分析"，"语法分析"，"关联"] 这个列表作为我的句子，它们都在 V_T 之中，属于 V_T^* ，所以说这个列表就是我的句子。

具体文法的完整定义如下：

$G = \{S,\ P,\ V_T,\ V_N\}$

$S =$ <Question>

$P = \{$

<Question> –> <类别> & <学科>

<类别> –> <词与词的关联> | <词与词之间的推导> | <对于单个词的解释>

<学科> –> <数据结构> | <计算机组成原理> | <编译原理> | <其他相关知识>

<对于单个词的解释> –> <只识别到一个词>

<词于词的关联> –> <识别到两个及以上> & "关系" | "关联" | "像" | "相关性" | "有关" | ("比"&"更")

<词与词的推导> –> <识别到两个及以上> & "推导" | "怎么从" & "到" | "延续性" | "转化" | "变成"

<只识别到一个词> –> count = 1

<识别到两个及以上> –> count >= 2

<数据结构> –> "栈" | "图" | "队列" | "搜索" | "排序" | "红黑树"....

<计算机组成原理> –> "寄存器组" | "控制器" | "冯诺依曼体系"....

<编译原理> –> "文法" | "词法" | "语法" | "语义 | **句柄 | 0 型文法 | 1 型文法**"....

<其他相关知识> –> "语法糖" | "语法树" | "API 设计" | "跨平台"....

}

$V_T = \{$ 栈，队列，树，图... $\} \cup \{$ "关系"， "关联"， "像"， "相关性"， "有关"，"比"&"更"， "推导" ， "怎么从" & "到"， "延续性"， "转化"， "变成" $\} \cup \{$ count $\}$

$V_N = \{$ <类别>，<学科>，<词与词的关联>，<词与词之间的推导>，<对于单个词的解释>，<数据结构>，<计算机组成原理>，<编译原理>，<其他相关知识> $\}$

> 可见 P 左边只有一个非终止符，故其为上下文无关的规则

三、具体功能

> **框架复用**：沿用之前的主程序框架，集成新功能；（整体要求–作业 1 要求 1）

优化点：沿用了问答系统的文法，在句柄标注中继续使用

模块	使用类型	对于实验实现内容	对应作业要求
句柄标注与可视化模块	兼有专用（即计算机知识问答系统）与通用 （两个类型选择）	文法句柄基础模块（讲解句柄定义与自动标注**句柄**）	作业 1 要求 2
		句柄可视化	作业 2 要求 1

文法判别与速查模块	通用	乔姆斯基**四类文法**自动判别与相关判断依据说明	作业 1 要求 3
		四类文法速查（交互对比表）和手动输入文法的分类（同上）	作业 2 要求 2
词法分析基础模块	通用	词法分析基础模块	作业 1 要求 4
专业课问答添加模块	专用	专业问答升级（添加句柄，四类文法，词法分析）	作业 1 要求 5

经过总结，把这次作业总结为 4 个模块

(一) 句柄标注与可视化模块

文法句柄基础模块（问答系统基础上识别句柄）

- 内置**算术表达式上下文无关文法，讲解句柄的定义**；
- 输入任意**合法句型，自动查找并标注句柄，输出解析结果**；

句柄可视化

- 实现基于句柄的最左归约步骤可视化，与最左推导+最右推导双向对应展示；
- 每一步归约自动标注当前句型的句柄，完整演示「推导→归约」流程；

1. 显示：对句柄定义的讲解（在规范规约之中的最左子串–实际上对于它的逆过程规范推导最右推导的最后一次右推导，也即右句型的最后一次右推导）

> **什么是句柄 (Handle)?**
>
> 在规范规约中，**句柄**是指右句型中匹配某个产生式右部，并且将其替换为左部非终结符后，代表了**最右推导，最后一步**（即最左规约的当前步）的那个子串。

2. 选择：对于基于我的问答系统（专用）还是 基于输入的通用文法（通用）进行 **（显示不同页面）**

◉ **A. 基于问答系统专用文法 (内置完整推导系统)**

○ **B. 基于通用文法 (需输入 G = {V_T, V_N, P, S})**

> 问答系统的整体架构

A 对于问答系统基础

3. 输入栏 1： 输入任何一个可识别句型，自动根据问答系统的文法（G）来进行自动查找并标注句柄（句型的标准遵守问答系统文法推导式风格）
4. 输入栏 2： 输入任何一个问题句子，自动判断规约过程中所有句型和其对对应的句柄，并以表格的形式来呈现。

输入栏 1：输入任何一个可识别句型，自动查找并标注句柄

count=1 栈 | 自动标注句柄

✅ 在句型 [count=1 栈] 中，
识别到的句柄是：[count=1]
应被规约为非终结符：[<对于单个词的解释>]

5. 最左规约可视化按钮+可视化区域：对于上述输入栏 2 的句子，可视化的显示最左规约的步骤，并也以树结构的形式显示剪枝的过程。同时在下面动态显示此时规约过程的句柄是哪个。

输入栏 2：输入问题句子，自动判断规约过程与句柄

请告诉我2型文法和3型文法的关系...

分析规约表格 | 最左规约可视化 | 最右推导可视化 | 最左推导可视化

规约步骤	当前右句型	识别的句柄	使用的产生式
1	count>=2 关系 2型文法 3型文法	count>=2 关系	<词与词的关联> -> count>=2 关系
2	<词与词的关联> 2型文法 3型文法	<词与词的关联>	<类别> -> <词与词的关联>
3	<类别> 2型文法 3型文法	2型文法 3型文法	<编译原理> -> 2型文法 3型文法
4	<类别> <编译原理>	<编译原理>	<学科> -> <编译原理>
5	<类别> <学科>	<类别> <学科>	<Question> -> <类别> <学科>
6	<Question>	<Question>	S -> <Question>
7	S	-	规约完成 (到达开始符)

注：专用模式下，系统已直接通过 Page1 的 AST 生成算法完成了目标句子的识别与建树。

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[count>=2 关系]，正将其归约为：[<词与词的关联>]

```
[S]
└── []
    ├── [<类别>]
    │   └── [<词与词的关联>]
    │       ├── count>=2
    │       └── 关系
    └── [<学科>]
        └── [<编译原理>]
            ├── 2型文法
            └── 3型文法
```

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[<词与词的关联>]，正将其归约为：[<类别>]

```
[S]
└── []
    ├── [<类别>]
    │   └── <词与词的关联>
    └── [<学科>]
        └── [<编译原理>]
            ├── 2型文法
            └── 3型文法
```

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[2型文法 3型文法]，正将其归约为：[<编译原理>]

```
[S]
└── []
    ├── <类别>
    └── [<学科>]
        └── [<编译原理>]
            ├── 2型文法
            └── 3型文法
```

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[<编译原理>]，正将其归约为：[<学科>]

```
[S]
└── []
    ├── <类别>
    └── [<学科>]
        └── <编译原理>
```

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[<类别> <学科>]，正将其归约为：[<Question>]

```
[S]
└── []
    ├── <类别>
    └── <学科>
```

动画播放：最左规约 (剪枝过程)

当前右句型句柄：[<Question>]，正将其归约为：[S]

```
[S]
└── <Question>
```

动画播放：最左规约 (剪枝过程)

🎉 **规约完成，到达开始符！ (演示结束)**

```
S
```

6. 最右推导可视化按钮+可视化区域：对于上述输入栏 2 的句子，可视化的显示最右推导的步骤，并也以树结构的形式显示构造的过程。同时在下面动态显示此时构造过程的句柄是哪个。（上面的逆过程）

分析规约表格 | 最左规约可视化 | 最右推导可视化 | 最左推导可视化

动画播放：最右推导 (建树过程)

🎉 **最右推导完成！ (演示结束)**

```
[S]
└── []
    ├── [<类别>]
    │   └── [<词与词的关联>]
    │       ├── count>=2
    │       └── 关系
    └── [<学科>]
        └── [<编译原理>]
            ├── 2型文法
            └── 3型文法
```

7. 最左推导可视化按钮+可视化区域：对于上述输入栏 2 的句子，可视化的显示最左推导的步骤，并也以树结构的形式显示构造的过程。同时在下面动态显示此时构造过程的句柄是哪个。

分析规约表格 | 最左规约可视化 | 最右推导可视化 | 最左推导可视化

动画播放：最左推导 (建树过程)

🎉 **最左推导完成！ (演示结束)**

```
[S]
└── []
    ├── [<类别>]
    │   └── [<词与词的关联>]
    │       ├── count>=2
    │       └── 关系
    └── [<学科>]
        └── [<编译原理>]
            ├── 2型文法
            └── 3型文法
```

B 对于通用文法

3. 输入栏：输入有穷文法（类似于）

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

○ **A. 基于问答系统专用文法 (内置完整推导系统)**

◉ **B. 基于通用文法 (需输入 G = {V_T, V_N, P, S})**

定义通用文法 G = {V_T, V_N, P, S}

V_T (终结符): id + * ( )

V_N (非终结符): E T F

S (开始符): E

P (产生式集合):

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

4. 对于上述有穷文法设置和问答系统一致的两个输入栏和三个可视化区域

输入栏 1：输入任何一个可识别句型，自动查找并标注句柄

id + id * id

自动标注句柄

✅ 在句型 [id + id * id] 中，
识别到的句柄是：[id]
应被规约为非终结符：[F]

输入栏 2：输入问题句子，自动判断规约过程与句柄

id * id + id

分析规约表格 | 最左规约可视化 | 最右推导可视化 | 最左推导可视化

规约步骤	当前右句型	识别的句柄	使用的产生式
1	id * id + id	id	F -> id
2	F * id + id	F	T -> F
3	T * id + id	id	F -> id
4	T * F + id	* F	+ -> * F
5	T + id	T	E -> T
6	E + id	id	F -> id
7	E + F	F	T -> F

(二) 文法判定与速查模块

乔姆斯基文法自动判定模块（通用的文法判断模块）

- 实现输入任意文法产生式，自动判定属于0型/1型/2型/3型文法；
- 输出判定依据（对应文法的规则特征）；

四类文法速查（文法速查模块）

- 制作0/1/2/3型文法交互式对比表（规则、限制、典型例子、应用场景）；
- 支持**手动输入文法**，快速验证分类结果，强化知识点记忆；

交互式对照表

1. 0/1/2/3 型文法交互式对照表（选择不同的规则显示不同文法的 基本规则、限制、典型示例以及该型文法的应用场景，多选时以对照表格的形式呈现）

乔姆斯基文法自动判定与速查

四类文法交互式对照表

请选择需要对比的文法类型： ☑ **0型 (短语结构)** ☑ **1型 (上下文有关)** ☑ **2型 (上下文无关)** ☑ **3型 (正规文法)**

文法级别	基本规则	核心限制	典型示例	应用场景 (对应自动机)
0型 (无限制文法)	α → β	α 中至少包含一个非终结符	a S b → a A b	图灵机 (Turing Machine)、自然语言处理底层
1型 (上下文有关)	α A β → α γ β	产生式右部长度 ≥ 左部长度 (|α| ≤ |β|)	a B c → a b c c	线性有界自动机 (LBA)、特定语境句法分析
2型 (上下文无关)	A → β	左部必须是单个非终结符 (A ∈ VN)	S → a S b | ab	下推自动机 (PDA)、编程语言语法分析(AST构建)
3型 (正规文法)	A → aB 或 A → a	右部最多一个非终结符，且只能全在最右(或最左)	A → a B | c	有限自动机 (DFA/NFA)、词法分析(正则表达式)

文法产生式

2. 文法产生式（即规则 P 的元素）的输入栏（自动判定属于哪一型文法，并且输出判定依据）

文法产生式自动判定工具

输入文法产生式 (以空格分隔符号，用大写字母表示非终结符)，系统将自动判断其所属的最高型文法。

```
a S b -> a A b
A -> c
```

自动判定分类

判定结果： **1型 (上下文有关文法)**

判定依据： 产生式左部包含多个符号，但所有产生式满足左部长度 ≤ 右部长度。

文法产生式自动判定工具

输入文法产生式 (以空格分隔符号，用大写字母表示非终结符)，系统将自动判断其所属的最高型文法。

```
S -> a A b
A -> c
```

自动判定分类

判定结果： **2型 (上下文无关文法)**

判定依据： 所有产生式左部均为单一非终结符，但不满足3型的线性严格限制。

文法产生式自动判定工具

输入文法产生式 (以空格分隔符号，用大写字母表示非终结符)，系统将自动判断其所属的最高型文法。

```
S -> a A
A -> c
```

自动判定分类

判定结果： **3型 (正规文法)**

判定依据： 所有产生式左部为一个非终结符，且右部符合纯右线性或纯左线性特征（最多只包含一个非终结符且位于一侧端点）。

(三) 词法分析基础模块

词法分析基础模块（通用的词法分析模块）

- 定义计算机**专业词法单元**：关键字、标识符、常数、运算符、界符；
- 对简单算术表达式（如 `x = 5 + a * 10`）完成基础词法切分，输出拆分后的符号列表；

1. 定义单词符号：关键词，标识符，常数，运算符，界符
2. 输入简单算术表达式

3. 进行分拆各符号

输入简易算术或逻辑表达式

```
int x = 5 + a * 10;
```

执行词法切分

词法 Token 序列输出

<KW, int> <ID, x> <OP, => <NUM, 5> <OP, +> <ID, a> <OP, *> <NUM, 10> <SEP, ;>

序号	单词符号 (Lexeme)	词法类别 (Token Type)	二元组表示
1	int	关键字 (Keyword)	<KW, 'int'>
2	x	标识符 (Identifier)	<ID, 'x'>
3	=	运算符 (Operator)	<OP, '='>
4	5	常数 (Constant)	<NUM, '5'>
5	+	运算符 (Operator)	<OP, '+'>
6	a	标识符 (Identifier)	<ID, 'a'>
7	*	运算符 (Operator)	<OP, '*'>
8	10	常数 (Constant)	<NUM, '10'>
9	;	界符 (Delimiter)	<SEP, ';'>

自动扫描源代码，切分为五大类单词符号（Token）：关键字、标识符、常数、运算符、界符。

输入简易算术或逻辑表达式

```
float a = 1.564 * key
```

执行词法切分

词法 Token 序列输出

<KW, float> <ID, a> <OP, => <NUM, 1.564> <OP, *> <ID, key>

序号	单词符号 (Lexeme)	词法类别 (Token Type)	二元组表示
1	float	关键字 (Keyword)	<KW, 'float'>
2	a	标识符 (Identifier)	<ID, 'a'>
3	=	运算符 (Operator)	<OP, '='>
4	1.564	常数 (Constant)	<NUM, '1.564'>
5	*	运算符 (Operator)	<OP, '*'>
6	key	标识符 (Identifier)	<ID, 'key'>

```
const tokenRegexList = [
    { type: "关键字 (Keyword)", regex: /^(int|float|void|return|if|else|while|for|break|continue)\b/ },
    { type: "常数 (Constant)", regex: /^\d+(\.\d+)?/ },
    { type: "标识符 (Identifier)", regex: /^[a-zA-Z_][a-zA-Z0-9_]*/ },
    { type: "运算符 (Operator)", regex: /^(\+|-|\*|\/|==|!=|>=|<=|>|<|=|&&|\||\||!)/ },
    { type: "界符 (Delimiter)", regex: /^([{}()\[\];,])/ }
];
```

(四) 专业课问答升级

专业课问答升级（问答系统基础上添加第四类 json 数据）：新增「句柄、四类文法、词法分析」基础概念智能问答。

1. 对 JSON 知识库进行添加

```
"编译原理": {
    "对于单个词的解释": {
        "句柄": "在自底向上语法分析（如LR分析）中，句柄是当前右句型中的一个特殊子串。它对应某条产生式的右部，且对其进行归约后能够还原到前一步的右句型。通俗地说，句柄是当前分析栈顶所有可归约串中最左",
        "0型文法": "又称无限制文法。其产生式形式为 α → β，其中 α 至少包含一个非终结符。对生成式没有任何限制，递归可枚举集，能力最强但实际编译中几乎不使用，因为其分析不可判定。",
        "1型文法": "又称上下文有关文法。产生式形式为 αAβ → αγβ（γ非空），即必须保留A的上下文才能进行替换。描述需要上下文信息的语言，但分析复杂度高，编译中很少直接使用。",
        "2型文法": "又称上下文无关文法。产生式形式为 A → γ，即无论A在什么上下文中，都可以被γ替换。这是编译原理的核心，用于描述大多数编程语言的语法结构（如表达式、语句）。",
        "3型文法": "又称正规文法。产生式形式为 A → aB 或 A → a（右线性），或 A → Ba 或 A → a（左线性）。它与正则表达式等价，用于描述词法规则。",
        "正规式": "用于描述单词模式的形式化表示，支持三种基本运算：连接、选择（|）和闭包（*）。每一个正规式对应一个正规集，并可转化为等价的有限自动机。",
        "确定有限自动机": "简称 DFA。状态图的一个特例，对于每个状态和每个输入符号，有且只有一个确定的下一个状态。DFA 在任一时刻只有一个激活状态，易于程序模拟，执行效率高。",
        "非确定有限自动机": "简称 NFA。其特点是一个状态对于同一个输入符号，可能有多个转移路径，甚至存在 ε 转移（不消耗输入字符）。NFA 易于从正规式直接构造，但模拟效率较低。",
        "Thompson构造法": "将正规式转化为 ε-NFA 的标准算法。通过为每个子表达式构建小 NFA，然后使用 ε 边按规则连接，最终得到等价的大 NFA。",
        "子集构造法": "将 NFA 转化为 DFA 的算法。核心思想是 DFA 的每一个状态对应 NFA 的一个状态子集。通过计算 ε-闭包和转移函数，构建出等价的 DFA。",
        "DFA的最小化": "将 DFA 的状态数减少到最少的过程。使用 Hopcroft 算法或填表算法，基于 Myhill-Nerode 定理，将所有等价的状态合并，得到唯一的最小 DFA。"
    },
    "词与词的关联": {
        "LL文法-LR文法": "两者都是上下文无关文法的重要子集。LL文法用于自顶向下（从左到右）最左推导）的语法分析，如递归下降；LR文法用于自底向上（最右推导的逆过程）的分析。通常 LR 文法的表达能力和覆
        "中间代码生成-目标代码生成": "中间代码生成将语法树转化为机器无关的形式（如三地址码），便于进行跨平台的通用代码优化；目标代码生成则是将优化后的中间代码转化为特定硬件架构的机器码（涉及寄存器分
        "有限自动机-正规表达式": "正规表达式是描述单词模式的形式化语法，而有限自动机（DFA/NFA）是实现这些模式匹配的底层计算模型。在词法分析器生成时，通常会将正规表达式先转化为 NFA，再确定化为 DFA。
        "抽象语法树-符号表": "抽象语法树记录了程序的语法结构，而符号表记录了程序中所有标识符（如变量、函数）的属性、类型和作用域。在语义分析阶段，编译器会遍历抽象语法树，并不断查询和更新符号表以进行
        "词法分析-语法分析": "词法分析器将字符流转化为 Token 流，供语法分析器使用。词法分析器简化了语法分析器的输入复杂度，将底层字符处理的细节封装起来。语法分析器则基于 Token 流的类别和值进行更高
        "2型文法-3型文法": "2型文法（上下文无关）用于描述嵌套结构（如括号配对、if嵌套），能力更强但分析效率较低（通常 O(n^3)）。3型文法（正规文法）只能描述线性重复结构（如标识符、数字），能力弱但分
        "正规式-有限自动机": "正规式和有限自动机在描述能力上是等价的，它们是同一事物的不同表现形式。正规式更接近人类直觉的描述语言，而有限自动机更接近机器的执行模型。编译原理中常通过“正规式 -> NFA
        "DFA-NFA": "NFA 是 DFA 的理论超集（加上非确定性和 ε 边），更容易从正规式手工或自动构造出来。DFA 是 NFA 的一种特例，执行效率高但状态数可能呈指数级增长。两者通过子集构造法相互转化，实现了“
    },
    "词与词的推导": {
        "词法分析-语法分析": "通过正规表达式定义单词模式，构建有限自动机实现词法分析器；词法分析器输出的 Token 序列作为语法分析器的输入，语法分析器根据上下文无关文法构建抽象语法树。",
        "中间代码-目标代码": "中间代码生成阶段将抽象语法树转化为三地址码等中间表示形式，便于进行平台无关的优化；目标代码生成阶段则根据特定硬件架构的指令集，将优化后的中间代码转化为机器码。",
        "词法分析-正规文法-有限自动机": "词法分析的理论基础是正则语言。正规式是正则语言的代数描述，正规文法（3型）是正则语言的文法描述，有限自动机是正则语言的识别装置。三者等价，任何正规式都可转化为
        "2型文法-句柄-归约": "自底向上分析（如 LR 分析）的过程是规范归约的逆过程。规范归约每次归约的是最左直接短语，即“句柄”。分析器维护一个符号栈，当栈顶形成某个产生式的句柄时，就执行“归约”操作（
        "0型文法-1型文法-2型文法-3型文法": "这四种文法构成了乔姆斯基谱系，是计算机科学中描述形式语言能力的层级结构。从 0 型到 3 型，约束条件逐渐增强，描述能力逐渐减弱，但分析算法的确定性和效率逐渐
    }
},
```

在 course_cat.json 文件中扩充相关条目。（在编译原理大类下进行扩充）

2. 在页面 1 知识问答系统进行验证

计算机专业课智能学习助手

支持学科：数据结构、计算机组成原理、编译原理

请帮我解释一下句柄是啥？

提交解析

即时答案

在自底向上语法分析（如LR分析）中，句柄是当前右句型中的一个特殊子串。它对应某条产生式的右部，且对其进行归约后能够还原到前一步的右句型。通俗地说，句柄是当前分析栈顶所有可归约串中最左的那个，是"下一步必须归约的符号串"。

计算机专业课智能学习助手

支持学科：数据结构、计算机组成原理、编译原理

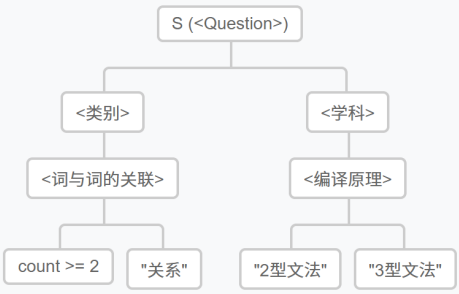
请帮我区分2型文法、3型文法

提交解析

即时答案

2型文法（上下文无关）用于描述嵌套结构（如括号配对、if嵌套），能力更强但分析效率较低（通常 O(n^3)）。3型文法（正规文法）只能描述线性重复结构（如标识符、数字），能力弱但分析效率极高（O(n)）。词法规则用3型文法，语法规则用2型文法，体现了"高效处理简单规则，强大能力处理复杂结构"的设计原则。

2. 句型推导过程 (EBNF)

```
S =>
↓
=> <类别> & <学科>
↓
=> <词与词的关联> & <编译原理>
↓
=> [解析规则: <词与词的关联> -> <识别到两个及以上> & "关系"] & [学科词汇: 2型文法, 3型文法]
↓
=> 终结符串: 2型文法 3型文法
```

3. 语法树可视化